1. Постановка задачи.

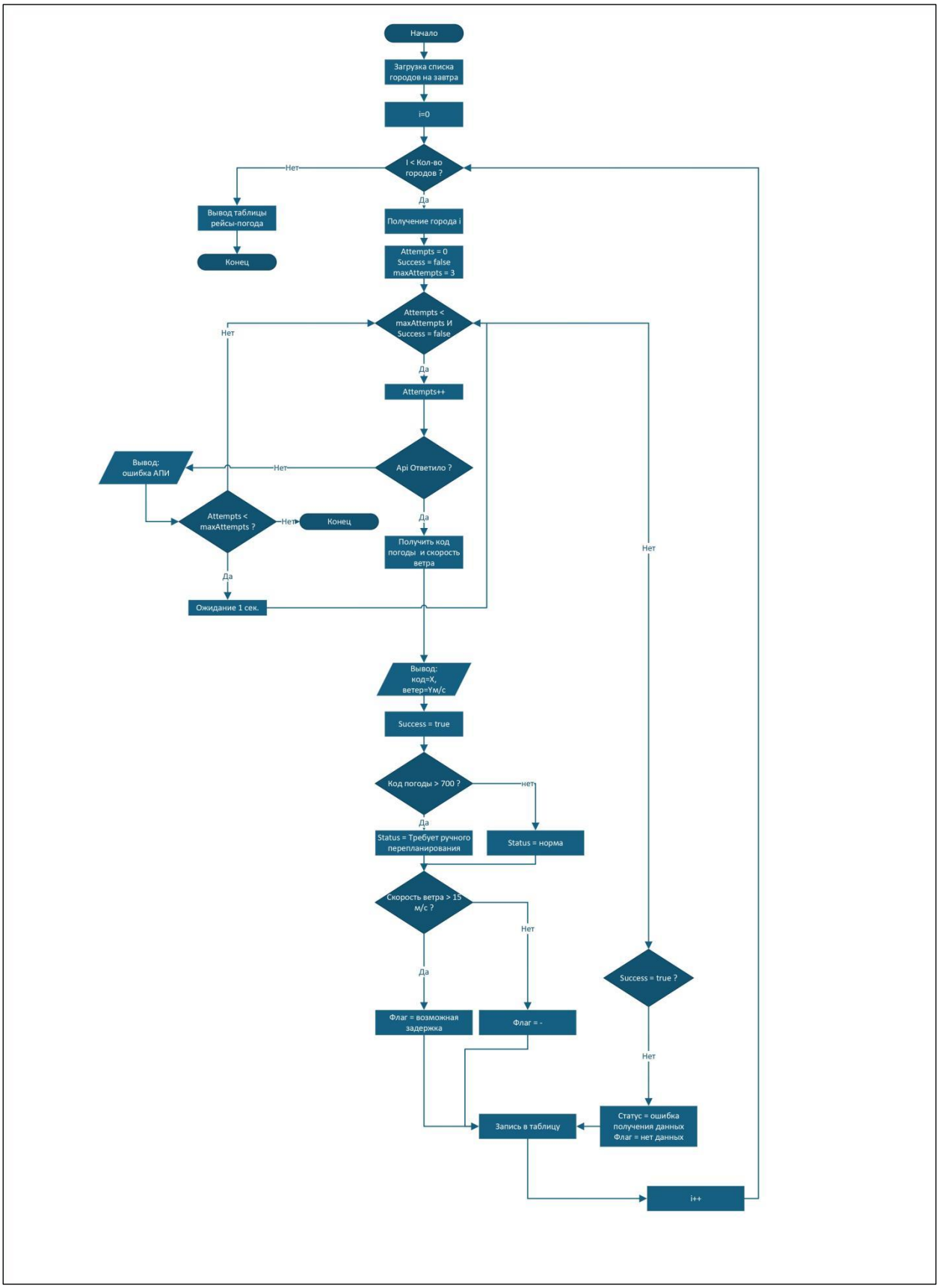
Грузоперевозочная компания хочет учитывать погоду при планировании маршрутов. Разработайте блок-схему процесса, который:

1. Берет из базы данных список городов отправления/назначения на завтра.
2. Для каждого города делает запрос к внешнему Weather API.
3. Если в ответе есть штормовое предупреждение (код погоды > 700), помечает рейс как "Требует ручного перепланирования".
4. Если скорость ветра > 15 м/с, устанавливает флаг "Возможная задержка".
5. Все результаты записывает в таблицу `Рейсы_Погода`.
6. Обрабатывает ошибки: если API не ответило, делает две повторные попытки с интервалом.

2. Контрольный пример.

Входные данные	**Значения**
Routes	101, "Москва", "Санкт-Петербург",
Выходные данные	**Значения**
Сообщение	Рейс 101 Москва → Санкт-Петербург Статус: Обычный Флаг задержки: Возможная задержка Погода: Город Москва: код 650,ветер 16,2 м/с; Город Санкт-Петербург: код 500, ветер 5,0 м/с;

3. Блок схема.



4. Текст программы.

```
int r = 0;
while (r < routes.Length)
{
    int routeId = routes[r].id;
    string fromCity = routes[r].from;
    string toCity = routes[r].to;

    string status = "Обычный";
    string delay = "Нет";
    string weatherInfo = "";

    string[] cities = { fromCity, toCity };
    int c = 0;
    while (c < cities.Length)
    {
        string city = cities[c];
        bool dataReceived = false;
        int weatherCode = 0;
        double windSpeed = 0;

        int attempt = 0;
        while (attempt <= 3)
        {
            int errorRand = rand.Next(1, 101);

            if (errorRand <= 20)
            {
                attempt++;
                continue;
            }
            weatherCode = rand.Next(100, 1000);
            windSpeed = rand.Next(0, 300) / 10.0;

            dataReceived = true;
            break;
        }

        if (dataReceived)
        {
            weatherInfo += $"Город {city}: код {weatherCode}, ветер {windSpeed} м/с; ";
            if (weatherCode > 700) status = "Требует ручного перепланирования";
```

```
            if (windSpeed > 15) delay = "Возможная задержка";
        }
        else
        {
            weatherInfo += $"Город {city}: ОШИБКА ПОЛУЧЕНИЯ ДАННЫХ; ";
            status = "Требует ручного перепланирования";
        }
        c++;
    }
    results.Add((routeId, fromCity, toCity, status, delay, weatherInfo));
    r++;
}
Console.WriteLine("Итоговая таблица рейсы погода:");
```

1. Постановка задачи.

Система умного склада получает показания с датчиков температуры.

Создайте блок-схему:

1 Датчик каждые 5 минут отправляет значение.

2 Если температура в норме (от +2 до +8 °C), значение просто пишется в лог.

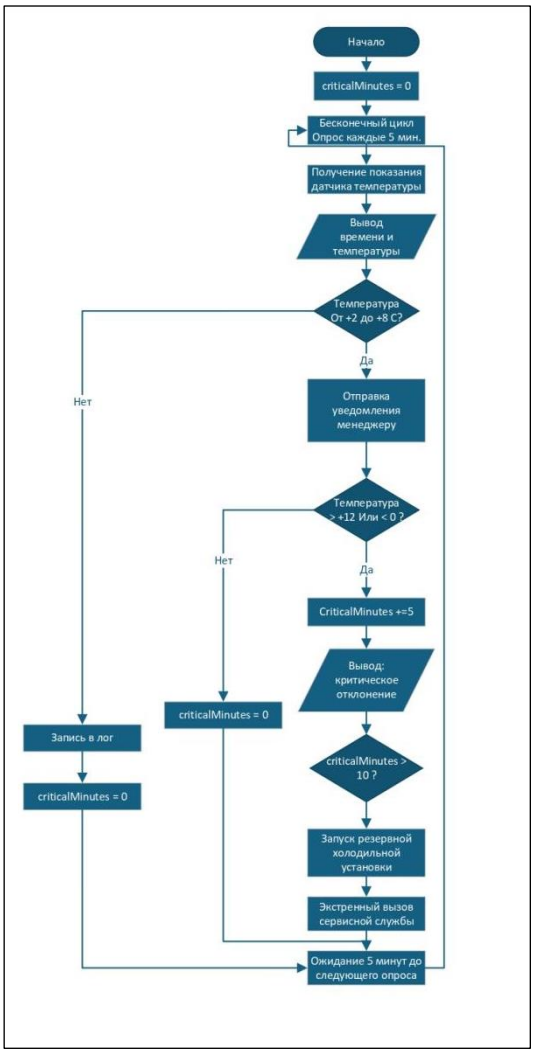
3 Если температура вышла за границы, система отправляет уведомление ответственному менеджеру в Telegram.

Если критическое отклонение длится более 10 минут (>+12 или <0 °C), система запускает резервную холодильную установку и отправляет экстренный вызов сервисной службе.

2. Контрольный пример.

Входные данные	**Значения**
T	5.0 C,9.0 °C
Выходные данные	**Значения**
Сообщение	"в норме", "вне нормы, но не критично"

3. Блок схема.


Начало
criticalMinutes = 0
Бесконечный цикл
Опрос каждые 5 мин.
Получение показания датчика температуры
Вывод времени и температуры
Температура От +2 до +8 С?
Да
Нет
Отправка уведомления менеджеру
Температура > +12 Или < 0 ?
Нет
Да
CriticalMinutes +=5
Вывод: критическое отклонение
criticalMinutes = 0
Запись в лог
criticalMinutes = 0
criticalMinutes > 10 ?
Запуск резервной холодильной установки
Экстренный вызов сервисной службы
Ожидание 5 минут до следующего опроса

4. Текст программы.

```
Console.WriteLine("Мониторинг температуры на складе...");
Console.WriteLine();

for (int i = 0; i < temperatures.Length; i++)
{
    Console.WriteLine("Показание " + (i + 1) + ": " + temperatures[i] + " °C");

    if (temperatures[i] >= 2 && temperatures[i] <= 8)
    {
        Console.WriteLine("Температура в норме. Запись в лог.");
    }
    else if (temperatures[i] > 8 || temperatures[i] < 2)
    {
        Console.WriteLine("Температура вне нормы! Отправка уведомления менеджеру в Telegram.");

        if (temperatures[i] > 12 || temperatures[i] < 0)
        {
            criticalMinutes += 5;
            Console.WriteLine("Критическое отклонение! Длительность: " + criticalMinutes + " минут");

            if (criticalMinutes >= 10 && !backupStarted)
            {
                Console.WriteLine("ЗАПУСК РЕЗЕРВНОЙ ХОЛОДИЛЬНОЙ УСТАНОВКИ!");
                Console.WriteLine("Экстренный вызов сервисной службы!");
                backupStarted = true;
            }
        }
        else
        {
            criticalMinutes = 0;
        }
    }

    Console.WriteLine();
    Thread.Sleep(500);
}

Console.WriteLine("Мониторинг завершен.");
```

1. Постановка задачи.

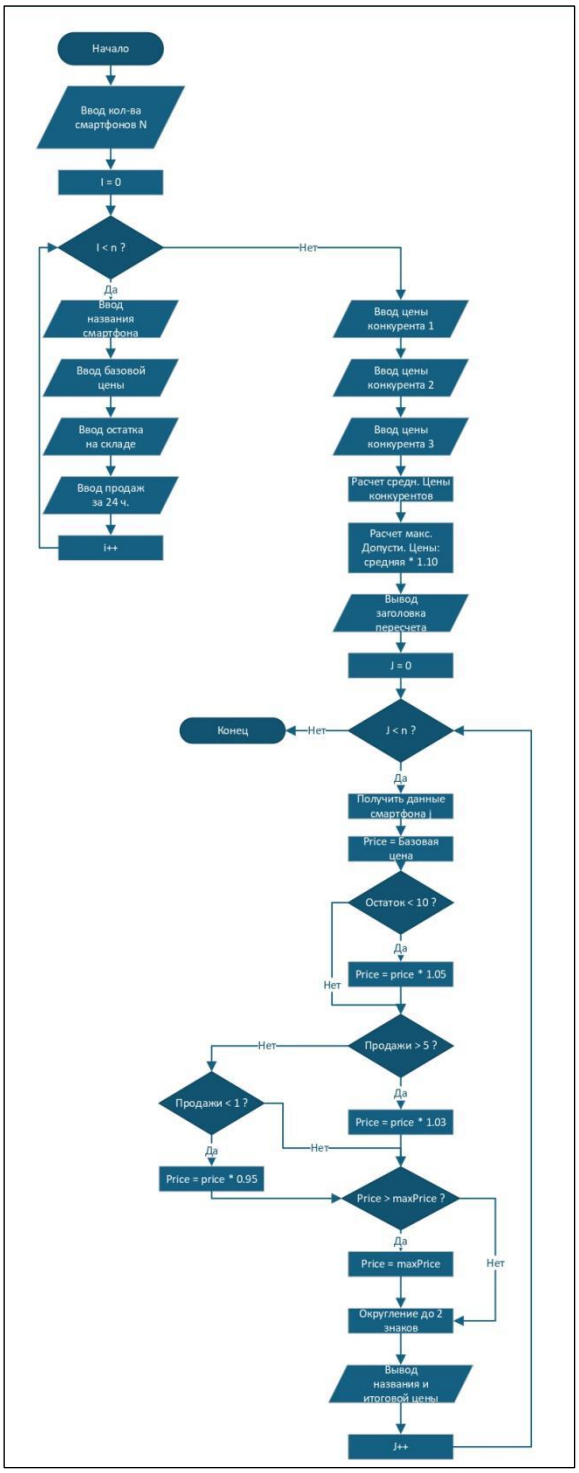
Для интернет-магазина электроники создайте блок-схему алгоритма, который раз в час пересчитывает цену на товары категории "Смартфоны" на основе:

1. Базовая цена из карточки товара.
2. Остаток на складе: если меньше 10 шт., цена повышается на 5%.
3. Скорость продаж за последние 24 часа: если высокая (>5 шт./день), цена повышается на 3%; если низкая (<1 шт./день) — понижается на 5%.
4. Конкуренты: система парсит (или получает из сервиса) цены 3 основных конкурентов. Наша цена не должна быть выше средней цены конкурентов более чем на 10%.
5. Итоговая цена округляется и применяется к товару.

2. Контрольный пример.

Входные данные	**Значения**
• Baseprice, stock, salespeed, competitorprice	80000,8->10,7->5,78000
Выходные данные	**Значения**
newprice	85800

3. Блок схема.


Начало
Ввод кол-ва смартфонов N
I = 0
I < n ?
Нет
Да
Ввод названия смартфона
Ввод базовой цены
Ввод остатка на складе
Ввод продаж за 24 ч.
i++
Ввод цены конкурента 1
Ввод цены конкурента 2
Ввод цены конкурента 3
Расчет средн. Цены конкурентов
Расчет макс. Допусти. Цены: средняя * 1.10
Вывод заголовка пересчета
J = 0
J < n ?
Нет
Конец
Да
Получить данные смартфона j
Price = Базовая цена
Остаток < 10 ?
Да
Price = price * 1.05
Нет
Продажи > 5 ?
Нет
Да
Price = price * 1.03
Продажи < 1 ?
Нет
Да
Price = price * 0.95
Price > maxPrice ?
Да
Price = maxPrice
Нет
Округление до 2 знаков
Вывод названия и итоговой цены
J++

4. Текст программы.

```
int i = 0;
while (i < phones.Length)
{
    Console.WriteLine("Товар: " + phones[i]);
    Console.WriteLine("Базовая цена: " + basePrices[i] + " руб");
    Console.WriteLine("Остаток на складе: " + stock[i] + " шт");
    Console.WriteLine("Продаж за 24ч: " + salesSpeed[i] + " шт");

    double newPrice = basePrices[i];

    if (stock[i] < 10)
    {
        newPrice *= 1.05;
        Console.WriteLine("Остаток < 10 шт: +5%");
    }

    if (salesSpeed[i] > 5)
    {
        newPrice *= 1.03;
        Console.WriteLine("Высокая скорость продаж: +3%");
    }
    else if (salesSpeed[i] < 1)
    {
        newPrice *= 0.95;
        Console.WriteLine("Низкая скорость продаж: -5%");
    }

    double avgCompetitorPrice = competitorPrices[i];
    double maxAllowedPrice = avgCompetitorPrice * 1.10;

    if (newPrice > maxAllowedPrice)
    {
        Console.WriteLine("Цена выше конкурентов на 10%. Корректировка...");
        newPrice = maxAllowedPrice;
    }

    newPrice = Math.Round(newPrice, 0);
    Console.WriteLine("Итоговая цена: " + newPrice + " руб");
    Console.WriteLine();

    i++;
}

Console.WriteLine("Цены обновлены.");
```

1. Постановка задачи.

Разработайте блок-схему модуля фрод-мониторинга для банковского приложения. Для каждой новой транзакции проверяется:

1. Геолокация: совпадает ли страна операции с местоположением телефона клиента?

2. Сумма и время: необычно ли большая сумма (>90% от среднего чека) или операция ночью (2:00-5:00)?

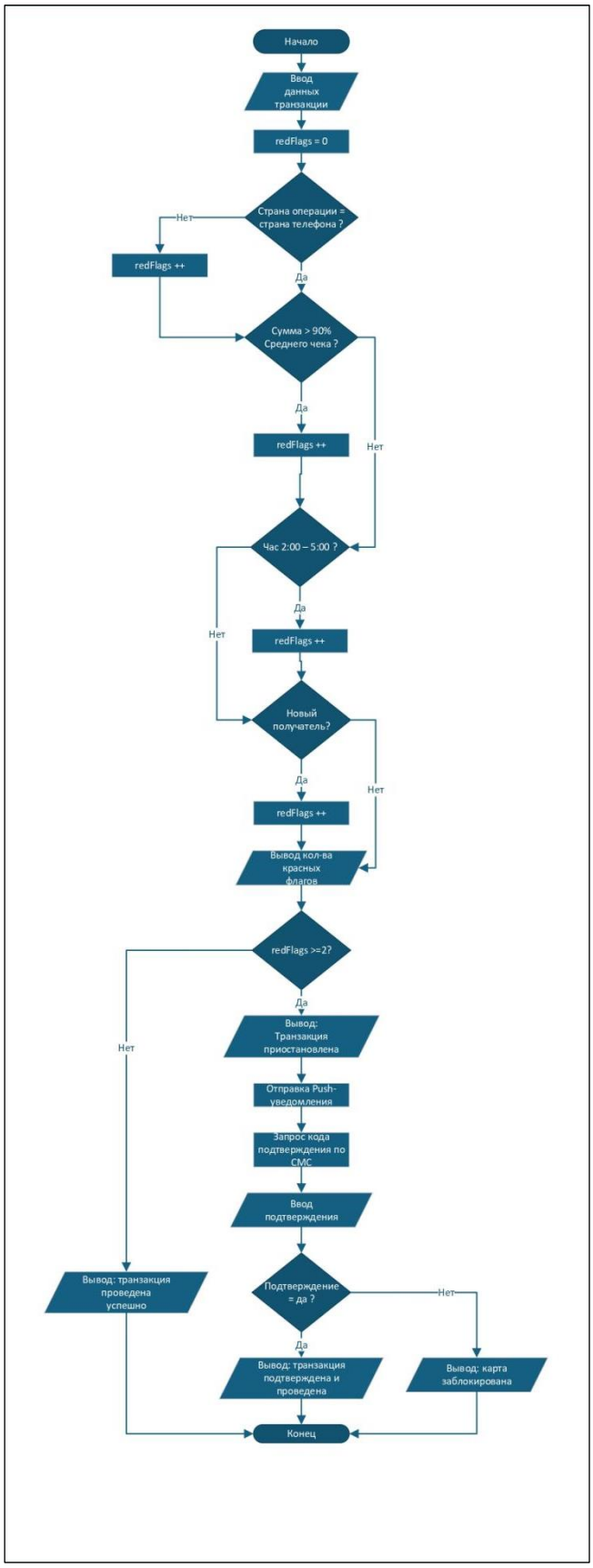
3. История: первая ли это операция с новым получателем?

4. Результат: Если 2 из 4 критериев "красные", транзакция приостанавливается, клиенту идет push-уведомление с запросом подтверждения по коду из SMS. Если клиент подтверждает — пропустить, если нет — заблокировать карту.

2. Контрольный пример.

Входные данные	**Значения**
• Сумма, Страна, время, новый получатель	5000, "Россия", 14:30, false
Выходные данные	**Значения**
redflag	0

3. Блок схема.


Начало
Ввод данных транзакции
redFlags = 0
Страна операции = страна телефона ?
Нет
redFlags ++
Да
Сумма > 90% Среднего чека ?
Да
redFlags ++
Нет
Час 2:00 – 5:00 ?
Да
Нет
redFlags ++
Новый получатель?
Да
Нет
redFlags ++
Вывод кол-ва красных флагов
redFlags >=2?
Да
Нет
Вывод: Транзакция приостановлена
Отправка Push-уведомления
Запрос кода подтверждения по СМС
Ввод подтверждения
Вывод: транзакция проведена успешно
Подтверждение = да ?
Нет
Да
Вывод: транзакция подтверждена и проведена
Вывод: карта заблокирована
Конец

4. Текст программы.

```
double avgCheck = 15000;

int i = 0;
while (i < amounts.Length)
{
    Console.WriteLine("Транзакция " + (i + 1) + ":");
    Console.WriteLine("Сумма: " + amounts[i] + " руб");
    Console.WriteLine("Страна операции: " + countries[i]);
    Console.WriteLine("Местоположение телефона: " + phoneLocations[i]);
    Console.WriteLine("Время: " + times[i]);
    Console.WriteLine("Новый получатель: " + newRecipient[i]);
    Console.WriteLine();

    int redFlags = 0;

    if (countries[i] != phoneLocations[i])
    {
        Console.WriteLine("Красный флаг: страна операции не совпадает с местоположением телефона");
        redFlags++;
    }

    if (amounts[i] > avgCheck * 0.9)
    {
        Console.WriteLine("Красный флаг: необычно большая сумма");
        redFlags++;
    }

    string[] timeParts = times[i].Split(':');
    int hour = Convert.ToInt32(timeParts[0]);
    if (hour >= 2 && hour <= 5)
    {
        Console.WriteLine("Красный флаг: операция ночью");
        redFlags++;
    }
```

```
    if (newRecipient[i])
    {
        Console.WriteLine("Красный флаг: первая операция с новым получателем");
        redFlags++;
    }

    Console.WriteLine("Красных флагов: " + redFlags);

    if (redFlags >= 2)
    {
        Console.WriteLine("ТРАНЗАКЦИЯ ПРИОСТАНОВЛЕНА!");
        Console.WriteLine("Отправка push-уведомления клиенту...");
        Console.WriteLine("Ожидание кода подтверждения из SMS...");

        Console.Write("Клиент подтверждает? (да/нет): ");
        string confirmation = Console.ReadLine().ToLower();

        if (confirmation == "да")
        {
            Console.WriteLine("Транзакция пропущена.");
        }
        else
        {
            Console.WriteLine("КАРТА ЗАБЛОКИРОВАНА!");
        }
    }
    else
    {
        Console.WriteLine("Транзакция разрешена.");
    }

    Console.WriteLine();

    i++;
```

1. Постановка задачи.

Создайте блок-схему для бронирования столика в ресторане через мобильное приложение.

1. Пользователь выбирает дату, время, количество гостей.

2. Система показывает доступные столы, учитывая:

- Свободные на выбранное время (основное правило).

- Предпочтение "у окна" (если отмечено в профиле).

- Особый кейс: если гостей > 8, система должна автоматически сдвинуть ближайшие два стола (проверить, свободны ли они оба).

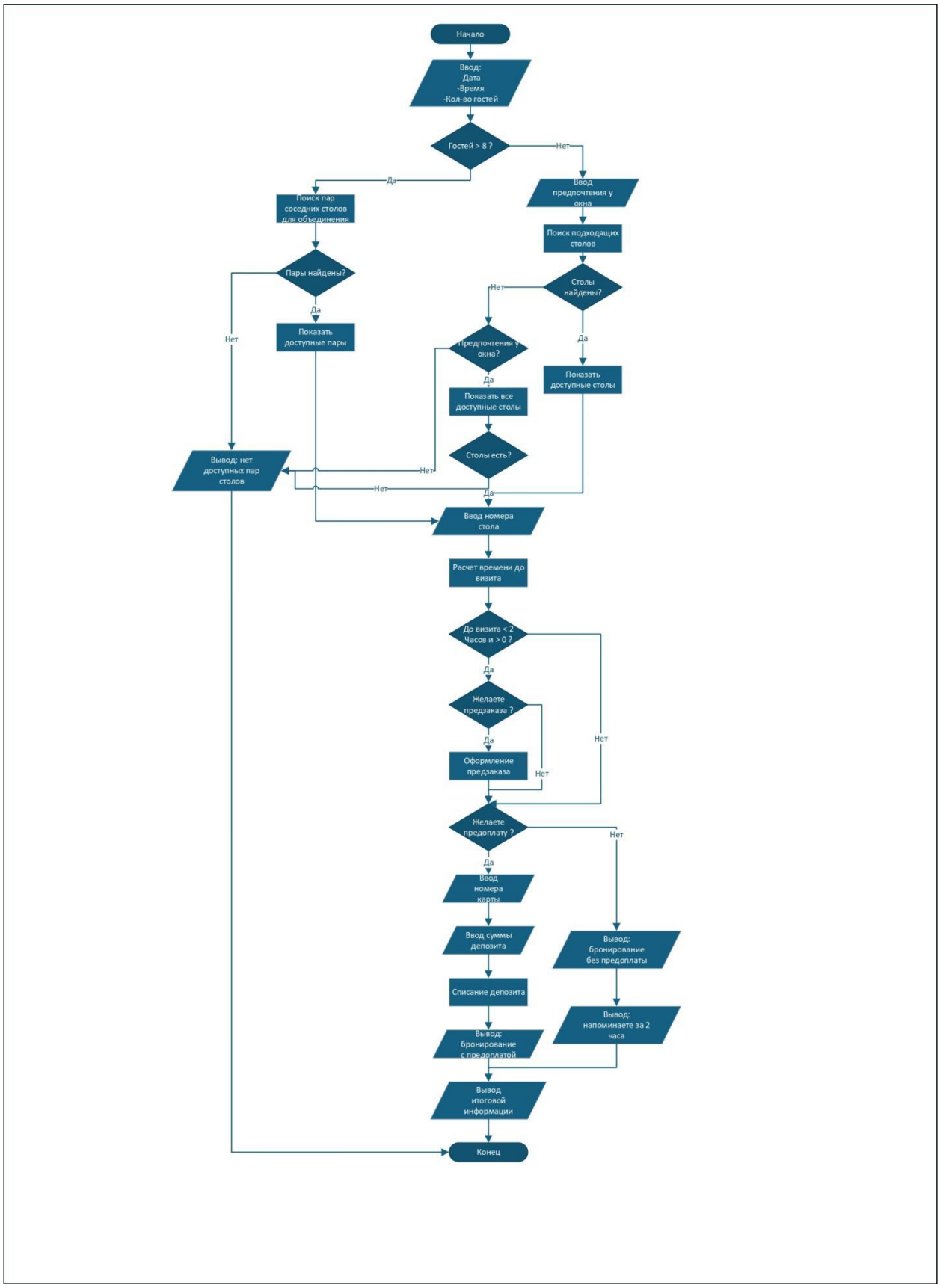
3. После выбора стола система предлагает предзаказ (если до визита меньше 2 часов) и выбор предоплаты (да/нет).

4. Ветвление: если с предоплатой — запрос реквизитов, списание депозита, подтверждение. Без предоплаты — просто подтверждение с напоминанием за 2 часа.

2. Контрольный пример.

Входные данные	**Значения**
• Дата, Время, Количество, гостей, Столик, Предоплата	15.03.2026,19:00,5,true
Выходные данные	**Значения**
Сообщение	"Бронирование потверждено с предоплатой"

3. Блок схема.


Начало
Ввод:
-Дата
-Время
-Кол-во гостей
Гостей > 8 ?
Нет
Да
Поиск пар соседних столов для объединения
Ввод предпочтения у окна
Поиск подходящих столов
Пары найдены?
Столы найдены?
Нет
Да
Показать доступные пары
Предпочтения у окна?
Показать доступные столы
Показать все доступные столы
Столы есть?
Вывод: нет доступных пар столов
Ввод номера стола
Расчет времени до визита
До визита < 2 Часов и > 0 ?
Желаете предзаказа ?
Оформление предзаказа
Желаете предоплату ?
Ввод номера карты
Ввод суммы депозита
Вывод: бронирование без предоплаты
Списание депозита
Вывод: напоминаете за 2 часа
Вывод: бронирование с предоплатой
Вывод итоговой информации
Конец

4. Текст программы.

```
    Console.WriteLine("Доступные столики:");
    int i = 0;
    while (i < tables.Length)
    {
        if (available[i])
        {
            Console.WriteLine((i + 1) + ". " + tables[i] + (byWindow[i] ? " (у окна)" : ""));
        }

        i++;
    }
    Console.WriteLine();

    if (guests > 8)
    {
        Console.WriteLine("Гостей больше 8. Поиск двух смежных столиков...");
        if (available[0] && available[1])
        {
            Console.WriteLine("Найдены столики 1 и 2. Объединение...");
        }
        else
        {
            Console.WriteLine("Нет подходящих смежных столиков.");
            return;
        }
    }

    Console.Write("Выберите номер столика: ");
    int tableChoice = Convert.ToInt32(Console.ReadLine()) - 1;

    if (tableChoice < 0 || tableChoice >= tables.Length || !available[tableChoice])
    {
        Console.WriteLine("Ошибка: неверный выбор столика.");
        return;
    }

    Console.WriteLine("Столик " + tables[tableChoice] + " выбран.");
```

```
int hoursBefore = 3;
if (hoursBefore < 2)
{
    Console.WriteLine("До визита меньше 2 часов. Предлагаем предзаказ.");
    Console.Write("Хотите сделать предзаказ? (да/нет): ");
    string preOrder = Console.ReadLine().ToLower();
}

Console.Write("Предоплата? (да/нет): ");
string prepayment = Console.ReadLine().ToLower();

if (prepayment == "да")
{
    Console.WriteLine("Запрос реквизитов для оплаты...");
    Console.WriteLine("Списание депозита...");
    Console.WriteLine("Бронирование подтверждено с предоплатой.");
}
else
{
    Console.WriteLine("Бронирование подтверждено без предоплаты.");
    Console.WriteLine("Напоминание будет отправлено за 2 часа до визита.");
}
```

1. Постановка задачи.

Нарисуйте блок-схему модуля Email-маркетинга, который формирует и отправляет рассылку.

На входе: ID шаблона письма и сегмент аудитории (например, "клиенты, купившие что-либо за последний месяц").

Система получает из БД список email и данных для персонализации (Имя, последний заказ и т.д.).

Для каждого клиента: генерируется персонализированный текст, создается задача на отправку во внешнем сервисе (SendGrid, Mailchimp).

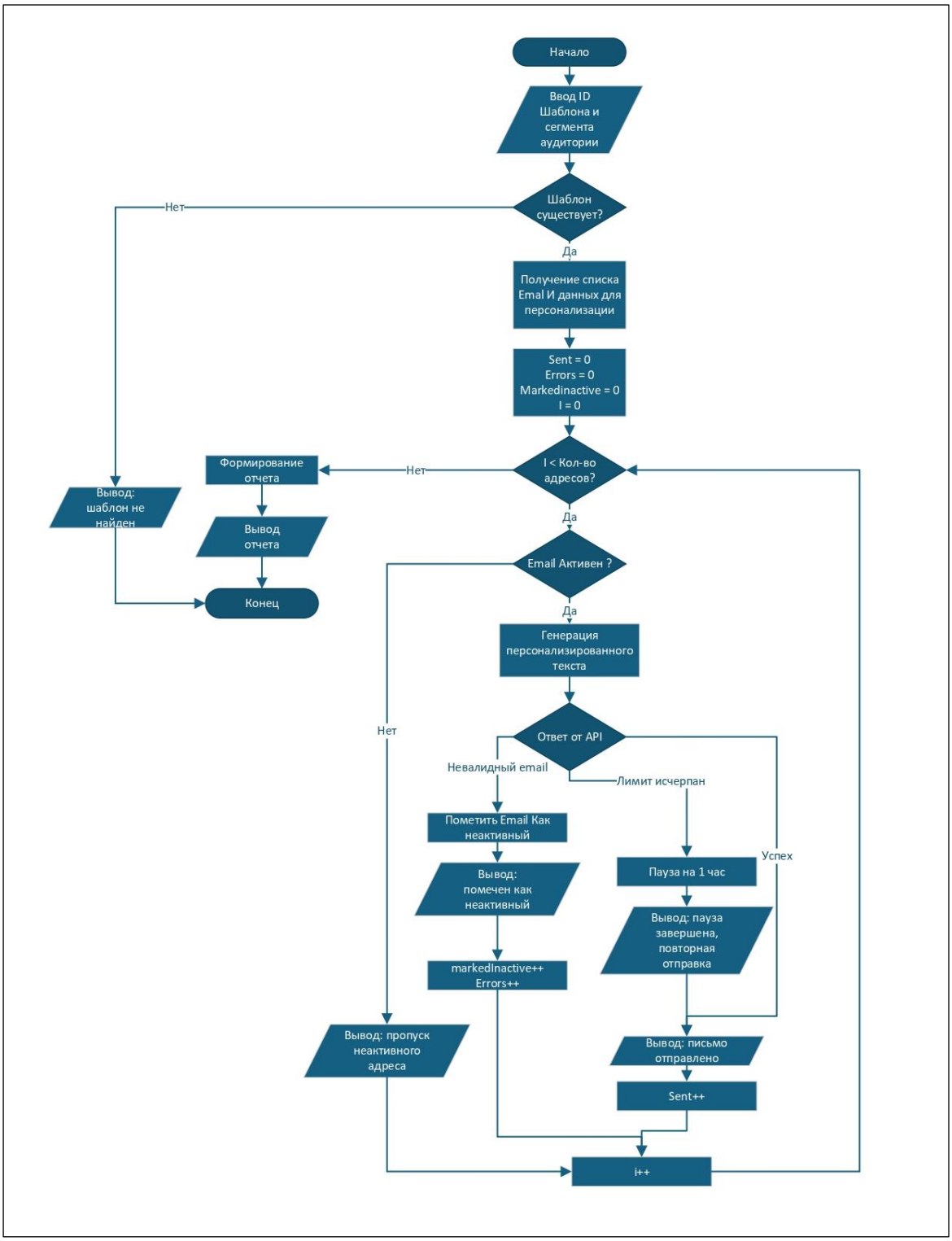
Обработка ошибок: Если сервис вернул ошибку "Невалидный email", адрес помечается в БД как "неактивный". При ошибке "Лимит исчерпан" процесс ставится на паузу на 1 час.

После отправки всем записывается отчет: сколько отправлено, сколько ошибок.

2. Контрольный пример.

Входные данные	**Значения**
• Email,names,lastorders	{ "ivan@example.com", "maria@example.com", "alex@example.com", "olga@example.com", "invalid@email" },{ "Иван", "Мария", "Алексей", "Ольга", "Пётр" },{ "Смартфон", "Наушники", "Ноутбук", "Планшет", "Часы" }
Выходные данные	**Значения**
Отправлено,Ошибок,Невалидных адресов	4,1,1

3. Блок схема.


Начало
Ввод ID Шаблона и сегмента аудитории
Шаблон существует?
Нет
Да
Получение списка Emal И данных для персонализации
Sent = 0
Errors = 0
Markedinactive = 0
I = 0
I < Кол-во адресов?
Нет
Да
Формирование отчета
Вывод отчета
Вывод: шаблон не найден
Конец
Email Активен ?
Нет
Да
Генерация персонализированного текста
Ответ от API
Невалидный email
Лимит исчерпан
Успех
Пометить Email Как неактивный
Вывод: помечен как неактивный
markedInactive++
Errors++
Пауза на 1 час
Вывод: пауза завершена, повторная отправка
Вывод: пропуск неактивного адреса
Вывод: письмо отправлено
Sent++
i++

4. Текст программы.

```
int sent = 0;
int errors = 0;
int invalid = 0;

int i = 0;
while (i < emails.Length)
{
    Console.WriteLine("Отправка письма " + (i + 1) + "/" + emails.Length);
    Console.WriteLine("Email: " + emails[i]);
    Console.WriteLine("Персонализация: " + names[i] + ", последний заказ: " + lastOrders[i]);

    Console.WriteLine("Генерация персонализированного текста...");
    Console.WriteLine("Создание задачи на отправку...");

    bool success = false;
    int attempts = 0;

    while (attempts < 3 && !success)
    {
        attempts++;

        if (emails[i].Contains("invalid"))
        {
            Console.WriteLine("Ошибка: невалидный email");
            invalid++;
            errors++;
            break;
        }

        if (i == 2 && attempts == 1)
        {
            Console.WriteLine("Ошибка: лимит исчерпан. Пауза 1 час...");
            Thread.Sleep(1000);
            continue;
        }
```

```
        Console.WriteLine("Письмо отправлено успешно.");
        sent++;
        success = true;
    }

    if (!success && !emails[i].Contains("invalid"))
    {
        Console.WriteLine("Не удалось отправить после " + attempts + " попыток.");
        errors++;
    }

    Console.WriteLine();

    i++;
}

Console.WriteLine("Отчет по рассылке:");
Console.WriteLine("Отправлено: " + sent);
Console.WriteLine("Ошибок: " + errors);
Console.WriteLine("Невалидных адресов: " + invalid);
```

1. Постановка задачи.

Создайте четкую блок-схему для runbook (инструкции) по восстановлению после сбоя.

1. Триггер: Система мониторинга обнаружила недоступность основного сервера БД.

2. Автоматическое оповещение ответственной команды.

3. Принятие решения: По данным логов определяем масштаб сбоя (полный отказ/повреждение данных). Если данные повреждены — запускаем восстановление.

4. Восстановление:

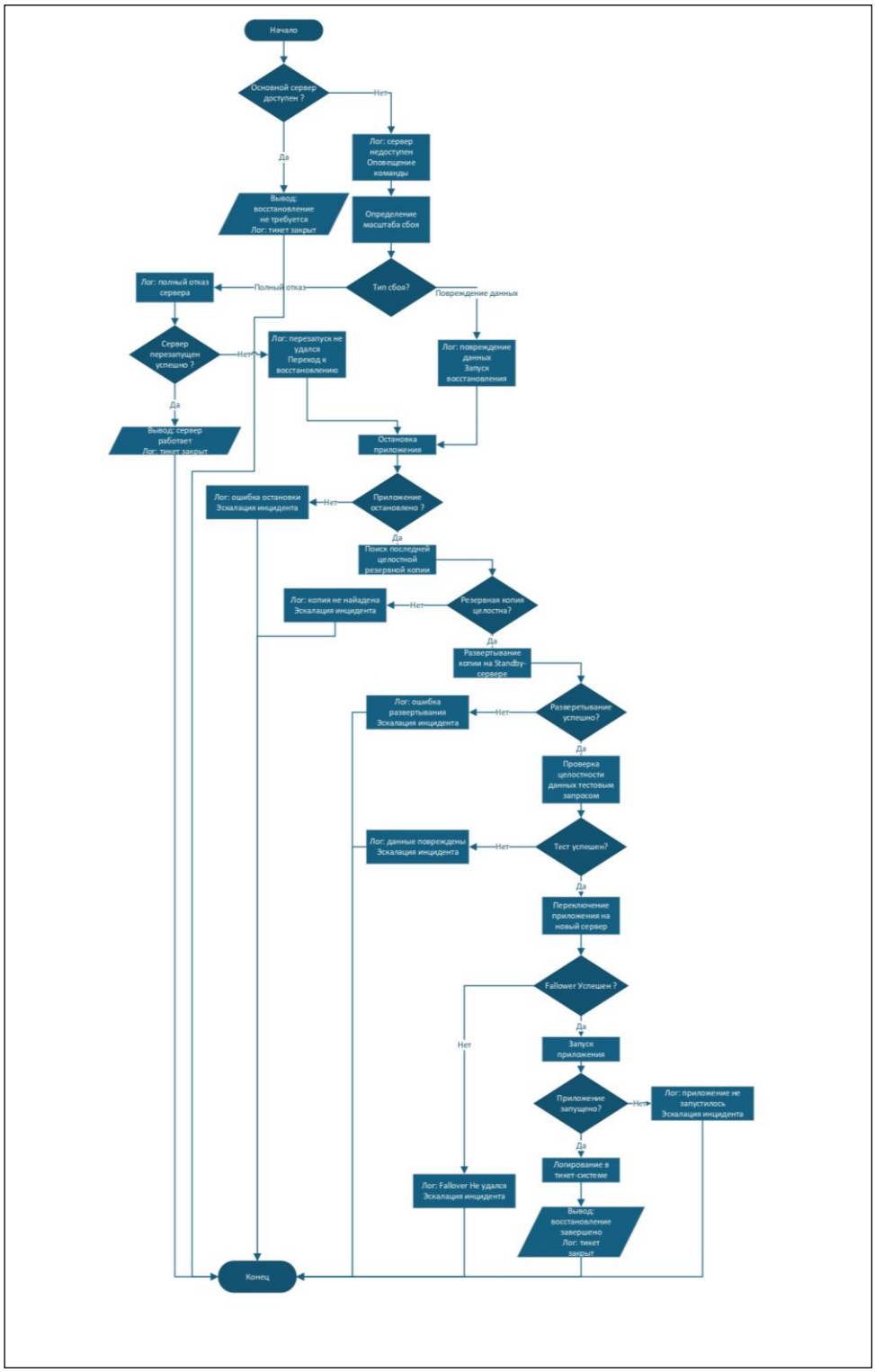
Остановить приложение -> Найти последнюю целостную резервную копию -> Развернуть её на standby-сервере -> Проверить целостность данных тестовым запросом -> Переключить (failover) приложение на новый сервер -> Запустить приложение.

5. Логирование: Каждый шаг и его результат фиксируются в тикет-системе.

2. Контрольный пример.

Входные данные	**Значения**
•	
Выходные данные	**Значения**
Сообщение	"Процедура востановления завершена"

3. Блок схема.


Начало
Основной сервер доступен ?
Нет
Лог: сервер недоступен Оповещение команды
Да
Вывод: восстановление не требуется Лог: тикет закрыт
Определение масштаба сбоя
Тип сбоя?
Полный отказ
Повреждение данных
Лог: полный отказ сервера
Сервер перезапущен успешно ?
Нет
Лог: перезапуск не удался Переход к восстановлению
Лог: повреждение данных Запуск восстановления
Да
Вывод: сервер работает Лог: тикет закрыт
Остановка приложения
Лог: ошибка остановки Эскалация инцидента
Нет
Приложение остановлено ?
Да
Поиск последней целостной резервной копии
Лог: копия не найдена Эскалация инцидента
Нет
Резервная копия целостна?
Да
Развертывание копии на Standby-сервере
Лог: ошибка развертывания Эскалация инцидента
Нет
Развертывание успешно?
Да
Проверка целостности данных тестовым запросом
Лог: данные повреждены Эскалация инцидента
Нет
Тест успешен?
Да
Переключение приложения на новый сервер
Fallower Успешен ?
Нет
Да
Запуск приложения
Приложение запущено?
Нет
Лог: приложение не запустилось Эскалация инцидента
Да
Логирование в тикет-системе
Лог: Fallover Не удался Эскалация инцидента
Вывод: восстановление завершено Лог: тикет закрыт
Конец

4. Текст программы.

```
Console.WriteLine("Процедура восстановления системы из резервной копии");
Console.WriteLine();

Console.WriteLine("Триггер: система мониторинга обнаружила недоступность основного сервера БД.");
Console.WriteLine("Лог: событие зарегистрировано в тикет-системе.");
Console.WriteLine();

Console.WriteLine("Автоматическое оповещение ответственной команды...");
Thread.Sleep(500);
Console.WriteLine("Команда уведомлена.");
Console.WriteLine("Лог: отправлено оповещение ответственной команде.");
Console.WriteLine();

Console.WriteLine("Анализ логов и метрик...");
Thread.Sleep(500);
Console.WriteLine("Определение масштаба сбоя (полный отказ / повреждение данных)...");
Thread.Sleep(500);

bool fullOutage = true;
bool dataCorrupted = true;

Console.WriteLine("Результат анализа: полный отказ сервера, данные повреждены.");
Console.WriteLine("Лог: принято решение выполнять восстановление из резервной копии.");
Console.WriteLine();

if (!dataCorrupted)
{
    Console.WriteLine("Данные не повреждены. Восстановление из резервной копии не требуется.");
    Console.WriteLine("Лог: выполнен перезапуск/переконфигурация без восстановления.");
    Console.WriteLine("Процедура завершена.");
    return;
}
```

```
Console.WriteLine("ЗАПУСК ПРОЦЕДУРЫ ВОССТАНОВЛЕНИЯ");
Console.WriteLine("Лог: процедура восстановления переведена в статус IN_PROGRESS.");
Console.WriteLine();

Console.WriteLine("Шаг 1: Остановка приложения...");
Thread.Sleep(500);
Console.WriteLine("Приложение остановлено.");
Console.WriteLine("Лог: приложение успешно остановлено.");
Console.WriteLine();

Console.WriteLine("Шаг 2: Поиск последней целостной резервной копии...");
Thread.Sleep(500);
Console.WriteLine("Найдена последняя целостная копия от 23.02.2026 22:00.");
Console.WriteLine("Лог: выбрана резервная копия для восстановления.");
Console.WriteLine();

Console.WriteLine("Шаг 3: Развертывание копии на standby-сервере...");
Thread.Sleep(1000);
Console.WriteLine("Копия развернута на standby-сервере.");
Console.WriteLine("Лог: восстановление БД на standby-сервере завершено.");
Console.WriteLine();

Console.WriteLine("Шаг 4: Проверка целостности данных...");
Thread.Sleep(500);
Console.WriteLine("Выполнение тестового запроса к восстановленной БД...");
Thread.Sleep(500);
Console.WriteLine("Данные целостны, тестовый запрос выполнен успешно.");
Console.WriteLine("Лог: проверка целостности данных завершена без ошибок.");
Console.WriteLine();

Console.WriteLine("Шаг 5: Переключение (failover) приложения на новый сервер...");
Thread.Sleep(500);
Console.WriteLine("Failover выполнен, приложение направлено на standby-сервер.");
Console.WriteLine("Лог: маршрутизация трафика переключена на новый сервер.");
Console.WriteLine();
```

```
Console.WriteLine("Шаг 6: Запуск приложения...");
Thread.Sleep(500);
Console.WriteLine("Приложение запущено на новом сервере.");
Console.WriteLine("Лог: приложение успешно запущено после восстановления.");
Console.WriteLine();

Console.WriteLine("Шаг 7: Логирование результатов...");
Console.WriteLine("Запись в тикет-систему: восстановление из резервной копии успешно завершено, сервис доступен.");
Console.WriteLine("Лог: тикет переведен в статус RESOLVED.");
Console.WriteLine();

Console.WriteLine("Процедура восстановления завершена.");
```

1. Постановка задачи.

Разработайте блок-схему ежеквартального фонового процесса.

Для таблицы `История_посещений_сайта` найти все записи старше 3 лет.

Проверить привязку: Есть ли у этих записей связь с открытыми судебными исками (через таблицу `Юридические_дела`)? Если да — пропустить. Если связи нет:* Перенести записи в таблицу `Архив_посещений` со статусом `archived`.

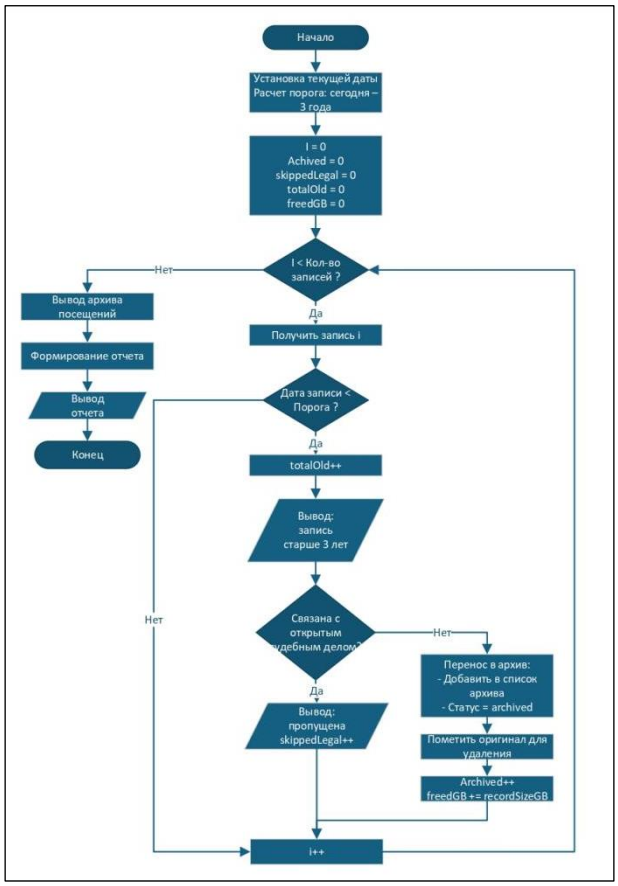
Удалить оригинальные записи из основной таблицы.

В конце процесса отправить отчёт администратору: сколько ГБ освобождено, сколько записей заархивировано, сколько исключено из-за юридических причин.

2. Контрольный пример.

Входные данные	**Значения**
• totalRecords,WithLegalCases	15000,12500
Выходные данные	**Значения**
Сообщение	"Процесс архивации завершен"

3. Блок схема.


Начало
Установка текущей даты
Расчет порога: сегодня – 3 года
I = 0
Achived = 0
skippedLegal = 0
totalOld = 0
freedGB = 0
I < Кол-во записей ?
Нет
Да
Вывод архива посещений
Формирование отчета
Вывод отчета
Конец
Получить запись i
Дата записи < Порога ?
Да
Нет
totalOld++
Вывод: запись старше 3 лет
Связана с открытым судебным делом?
Нет
Да
Перенос в архив:
- Добавить в список архива
- Статус = archived
Пометить оригинал для удаления
Archived++
freedGB += recordSizeGB
Вывод: пропущена skippedLegal++
i++

4. Текст программы.

```
Console.WriteLine("Алгоритм очистки/архивации устаревших данных");
Console.WriteLine();

Console.WriteLine("Поиск записей старше 3 лет в таблице История_посещений_сайта...");
int totalRecords = 150000;
Console.WriteLine("Найдено: " + totalRecords + " записей");
Console.WriteLine();

Console.WriteLine("Проверка связей с юридическими делами...");
int withLegalCases = 12500;
int withoutLegalCases = totalRecords - withLegalCases;
Console.WriteLine("Связаны с открытыми исками: " + withLegalCases + " записей (пропущены)");
Console.WriteLine("Без связей: " + withoutLegalCases + " записей (к архивации)");
Console.WriteLine();

Console.WriteLine("Архивация записей...");
Console.WriteLine("Перенос в таблицу Архив_посещений со статусом 'archived'...");

int archived = 0;
int batchSize = 10000;

int i = 0;
int batchNumber = 1;
while (i < withoutLegalCases)
{
    int currentBatch = Math.Min(batchSize, withoutLegalCases - i);
    Console.WriteLine("Пакет " + batchNumber + ": " + currentBatch + " записей");
    archived += currentBatch;

    i += batchSize;
    batchNumber++;
}
```

```
Console.WriteLine("Всего заархивировано: " + archived + " записей");
Console.WriteLine();

Console.WriteLine("Удаление оригинальных записей из основной таблицы...");
Console.WriteLine("Удалено: " + archived + " записей");
Console.WriteLine();

double gbFreed = 2.5;
Console.WriteLine("Отчет администратору:");
Console.WriteLine("Освобождено: " + gbFreed + " ГБ");
Console.WriteLine("Заархивировано записей: " + archived);
Console.WriteLine("Исключено из-за юридических причин: " + withLegalCases);
Console.WriteLine();

Console.WriteLine("Процесс архивации завершен.");
```